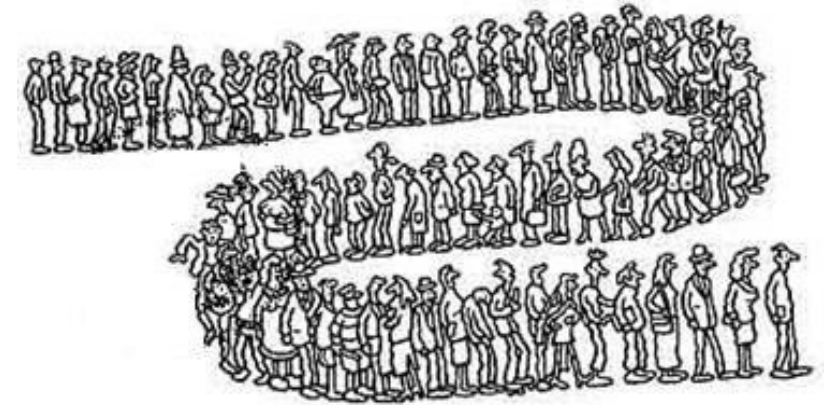


CS5239 Computer System Performance Analysis

L3: Introduction to Queueing Theory

Outline

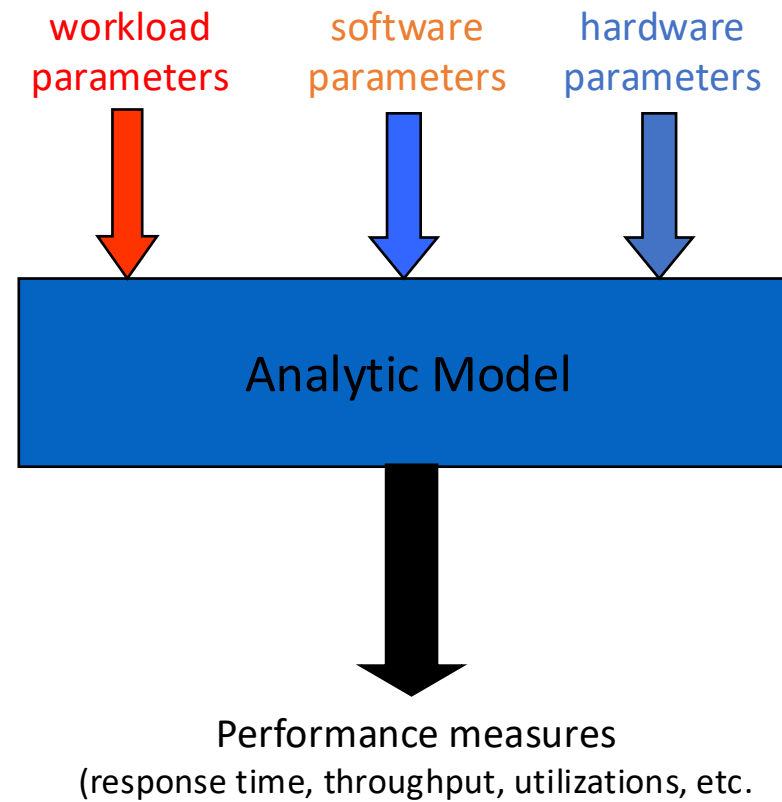
- Analytic models
- Stochastic processes
- Queueing theory
 - Queueing Networks
 - Basic Components of a Queueing System
 - Kendall Notation
 - Rules for all Queues
- Little's Law
- Summary



“It is very difficult to make an accurate prediction, especially about the future.”

- *Niels Bohr*

Pictorial Representation of an Analytic Model



Analytic Models

- Mathematical representation of a system
- **Advantages:**
 - More convenient to model a computer configuration than to actually assemble or gain access to a real one
 - Much simpler and quicker to alter a model than a real system
 - Timescales for running experiments can often be substantially reduced
 - Workload and system parameters can be varied, and the system subjected to extreme conditions that would not be possible using a real system
 - Actual process of building a model helps to gain a clearer understanding of how the real system functions

Analytic Models

- Disadvantages:
 - Expertise and time devoted to building, validating, and using the model
 - Model behaviour is an approximation of the real system
 - Details in the model, assumptions made, and result accuracy, etc. require careful analysis and interpretation

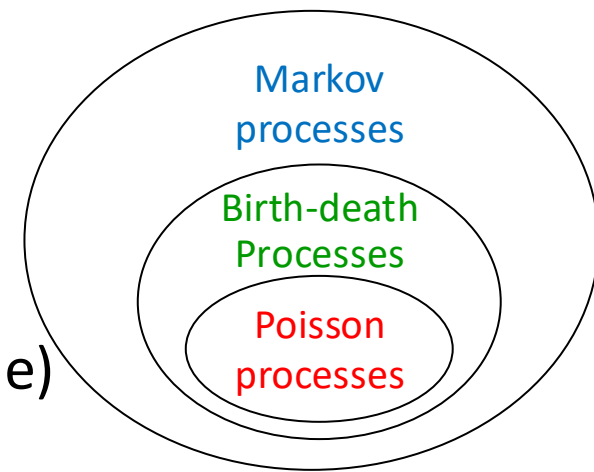
Stochastic Processes

- A stochastic process is a collection of random variables indexed by time, describing how a system evolves under uncertainty
 - A mathematical object usually defined as a **sequence of random variables** in a **probability space**, where the index of the sequence often has the interpretation of **time**
- Examples
 - Let $N(t)$ denotes the number of jobs at the CPU at time t . $N(t)$ is a discrete random variable. Why?
 - Collection of $N(t)$ for different values of t is a stochastic process
 - To specify its behavior, determine the probability mass function (pmf) for $N(t)$ at each possible value of t
 - Let W_n denotes the waiting time of job n at the CPU. W_n is a random variable (continuous). Why?
 - Collection of W_n is a stochastic process
 - Need to specify the probability density functions (pdf) for W_n for all n

Types of Stochastic Processes

- Based on their mathematical properties, stochastic processes can be
 - Random walks
 - Markov processes
 - Lévy processes
 - Gaussian processes
 - Renewal processes
 - Branching processes.

Subsets of Stochastic Processes



- Discrete-time or continuous-time stochastic processes
 - Finite and countable number of elements of the index set (time)
- Markov processes (Markov chains)
 - If the future states of a process are independent of the past and depend only on the present
- Birth-death processes
 - Discrete-space Markov processes where the transitions are restricted to neighboring states
- Poisson Processes
 - If the inter-arrival times are independent and exponentially distributed, the number of arrivals n over a given interval $(t, t + x)$ has a Poisson distribution. The arrival process is a Poisson process or a Poisson stream.

Markov Process

- Stochastic process describing a sequence of possible events in which the probability of each event depends only on the state attained in the previous (current) event
- Discrete-time Markov chain
 - A countably infinite sequence, in which the chain moves state at discrete time steps
- Continuous-time Markov chain
 - A continuous-time process
- Applied in biology, chemistry, economics, finance, information theory, physics, signal processing, speech processing, etc

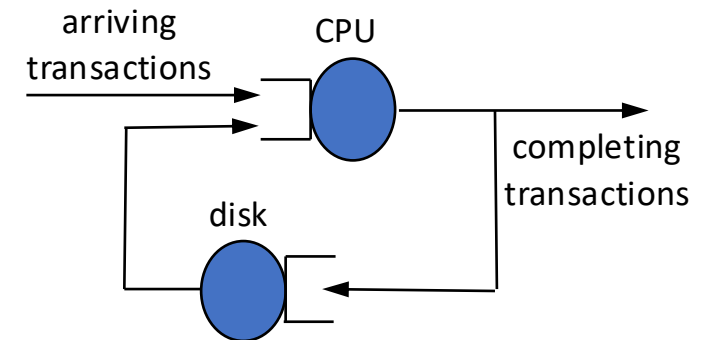
Markov Property

- To make predictions of the behavior of a system in the future, it suffices to consider only the present state of the system and not the past history
- Memoryless process

Key Approaches in Analytical Modeling

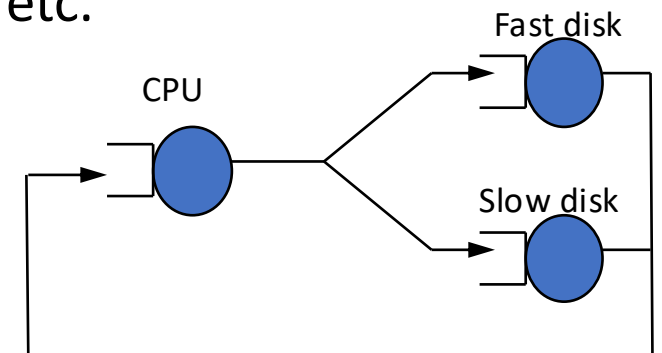
- **Queueing theory** - job arrivals and service times are probabilistic

- Single queue systems, queueing network systems
- Markovian and non-Markovian queueing models
- Examples: database server modeled as queueing networks



- **Operational analysis** - based on averages

- Measure number of job arrivals, number of completed jobs, busy time
- Operational quantities: utilization, response time, throughput, etc.
- Operational laws relate to the operational quantities
- Bottleneck and bounding analysis



Practical Example: Analytic Model Inputs And Outputs

A model is useful when its inputs correspond to quantities we can measure or forecast

Workload Inputs

- Arrival rate λ
- Request mix
- Think time
- Input size and variability

System Inputs

- Number of servers
- Service-time distribution
- Buffer capacity
- Routing and discipline

Predictions

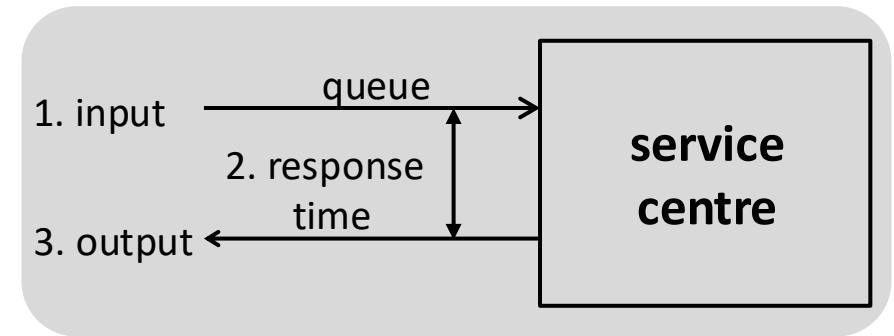
- Response time
- Throughput
- Utilization
- Queue length
- Stability or saturation

Practical check: every important model input should have a measurement, estimate, or stated assumption

Roadmap

Roadmap

- Review of Probabilities & Statistics (Lstats) – slides provided
- Queueing Theory (L3 & L4)
- Operational Analysis (L3 & L5)



How long will you spend queueing in your lifetime?

Me: “It will take two minutes”

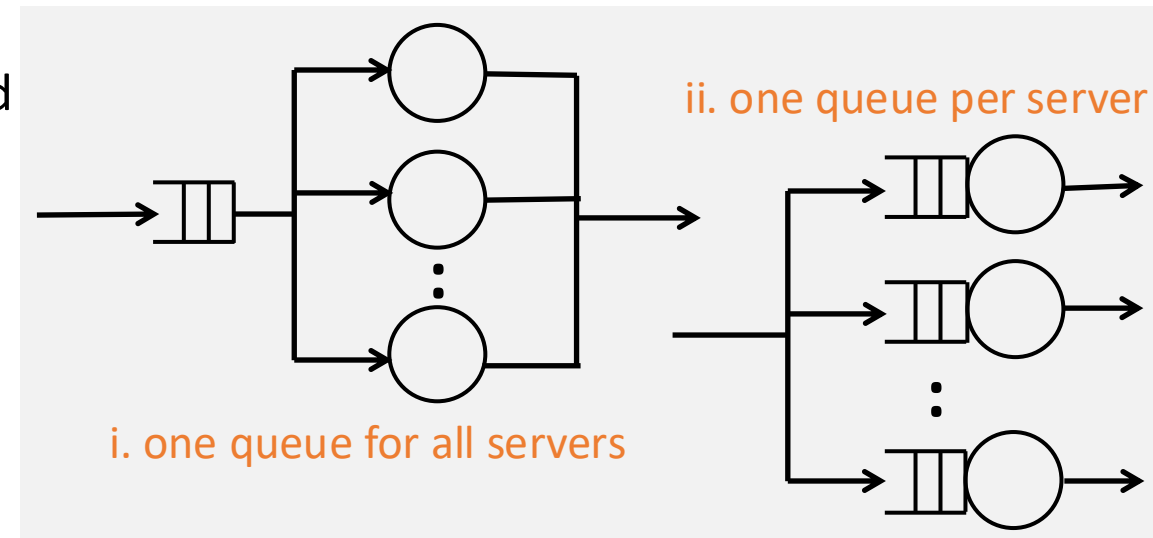
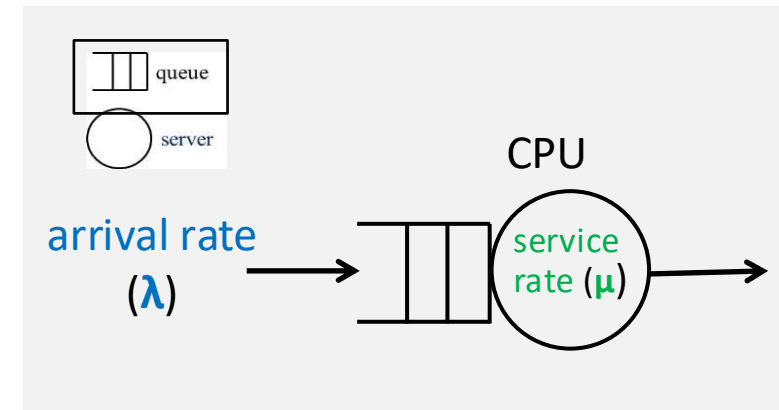
Britons spent almost **6 months** of their life queueing, according to new research.
- The Telegraph, 26 Mar 2009

The queue: λ is approaching service capacity



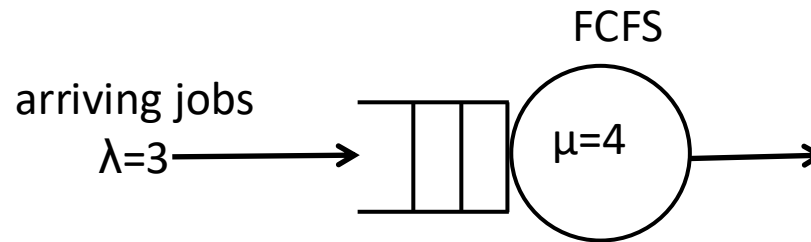
Performance Questions

1. What is the average time it takes a job to complete service?
2. What is the throughput of the system (number of jobs completed per unit time)?
3. If arrival rate is doubled ($\lambda \rightarrow 2\lambda$), do we do nothing or how much should μ increase?
4. If we need more server (CPU) capacity, what are our options?
 - buy a new server with the needed capacity
 - buy a few smaller servers that adds up to the required capacity
 - how do we organize these servers?
 - i. one queue for all servers
 - ii. one queue for each server
 - iii. does it matter?



Queueing Theory

- Study of queueing behavior in systems and networks
- In a queueing system, jobs arrive, are served and leave the system. Processes interact with each other, evolve over time and are of probabilistic nature - processes modeled as random variables
- A queueing network is made up of **servers** (also called **service centers**)
- The simplest example of a queueing network is the single-server network



queueing networks

```
graph TD; A[queueing networks] --> B[open<br/>finite buffer, infinite<br/>buffer]; A --> C[closed<br/>interactive, batch]
```

open

finite buffer, infinite
buffer

closed

interactive, batch

Practical Example: open Or closed

Choose the model based on how jobs enter and leave the system

Open System

External users can arrive and depart independently

Examples

Public web service
Network router
Online search service

Closed System

A fixed population cycles between thinking and submitting work

Examples

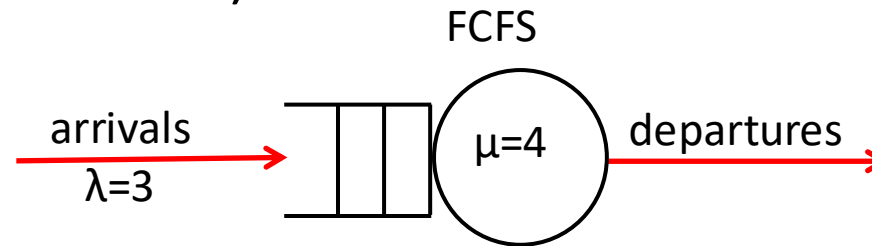
Interactive terminals
Fixed batch jobs
Fixed worker pool

Ask first: can new jobs enter from outside while existing jobs remain in the system?

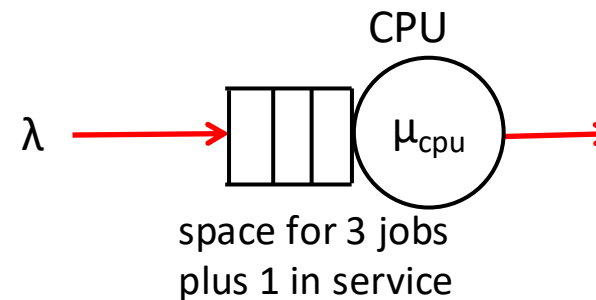
Open Queueing Networks

- External arrivals and departures - four examples of open networks

i. single-server system (infinite buffer)

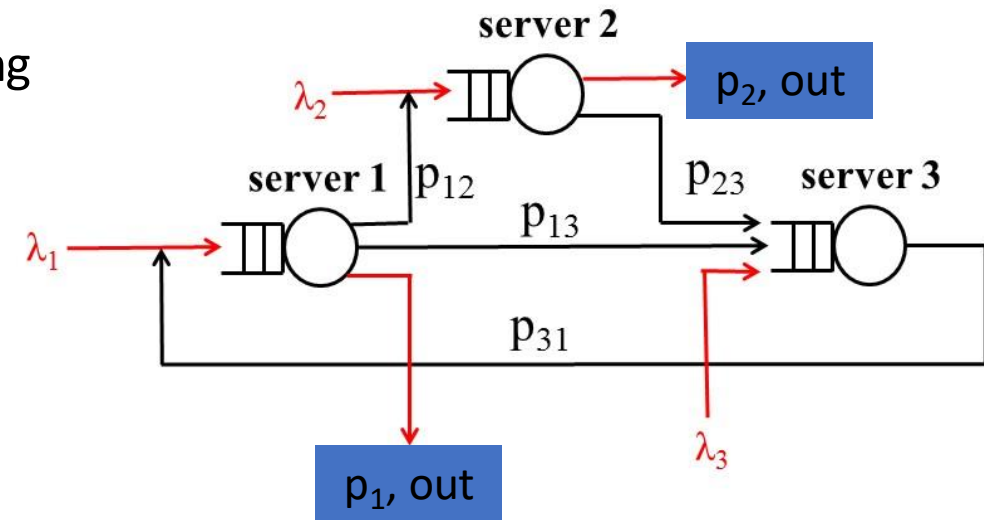


ii. single-server system with finite buffer

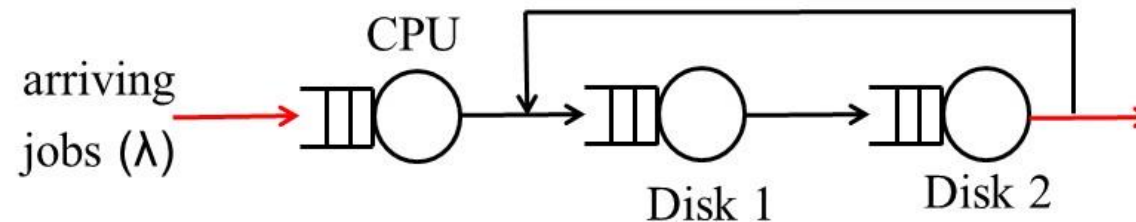


Open Queueing Networks

iii. network of queues with probabilistic routing



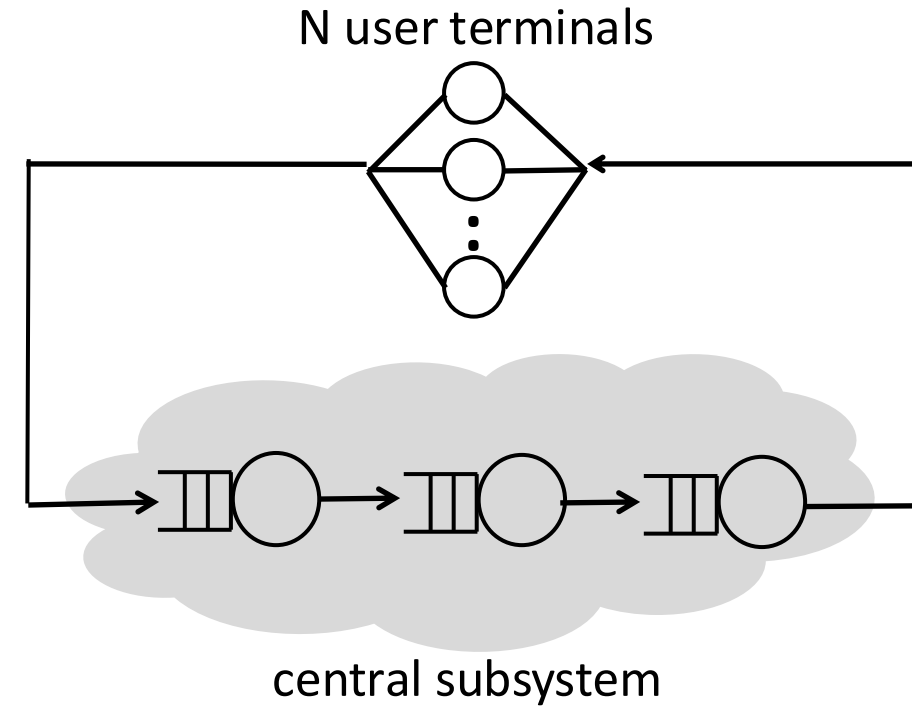
iv. network of queues with non-probabilistic routing



jobs followed a
predetermined route:
CPU, disk 1, disk2,
disk 1, disk 2 and out
(leave system)

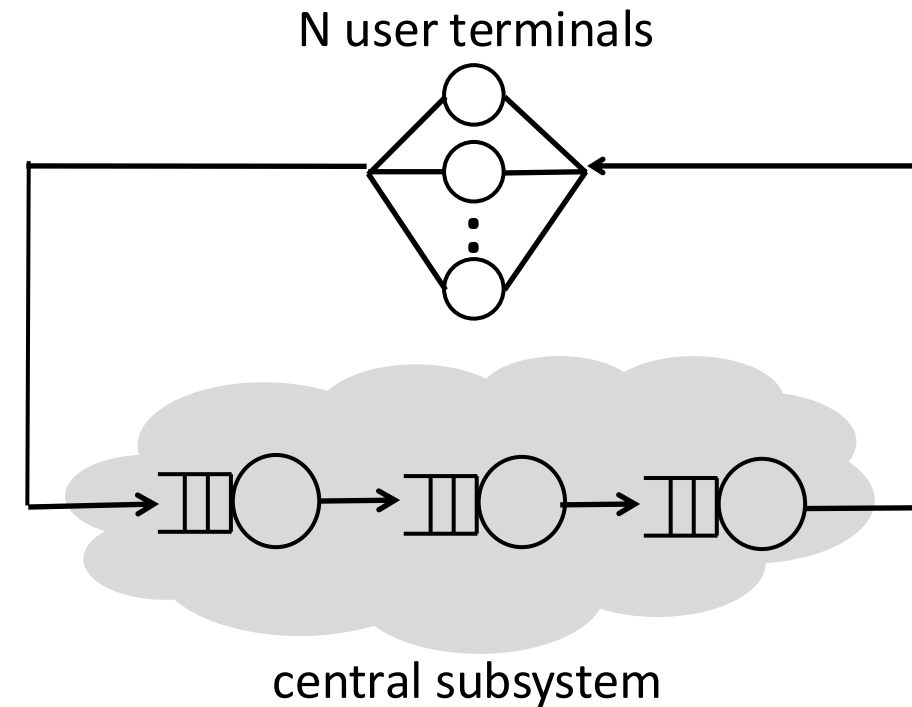
Closed Queueing Networks

- No external arrivals and departures
 - i. interactive (terminal-driven) systems



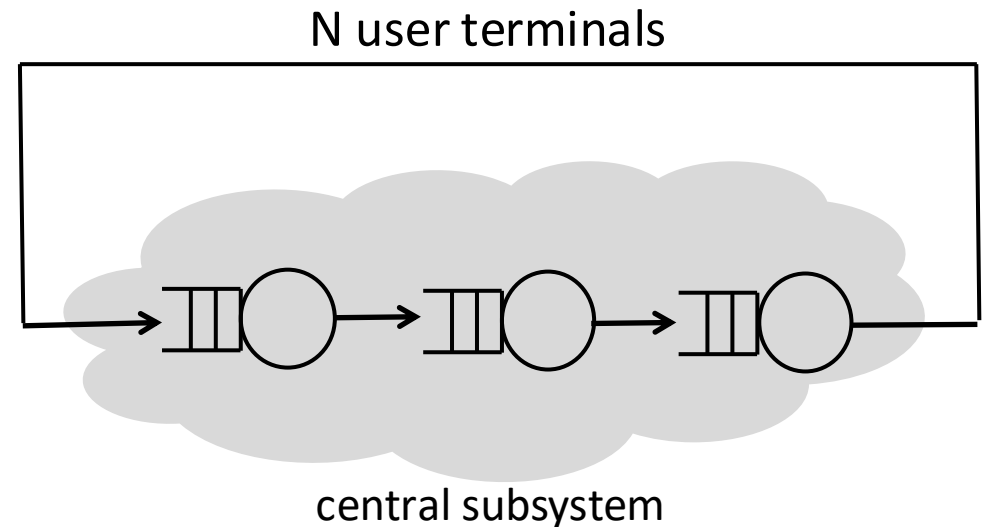
Interactive (Terminal-driven) Systems

- Support as many terminals (users) as possible while keeping response time low
- Terminals represent users who each send a job to the “central subsystem” and then wait for a response
- Central subsystem is a network of queues
- A user cannot submit the next job before the previous job returns (complete processing) and thus number of jobs in the system is fixed (equal to number of terminals, N , also called load or multiprogramming level)
- Think time (Z) – a random variable representing the time at each terminal between receiving the result of one job and sending out the next job



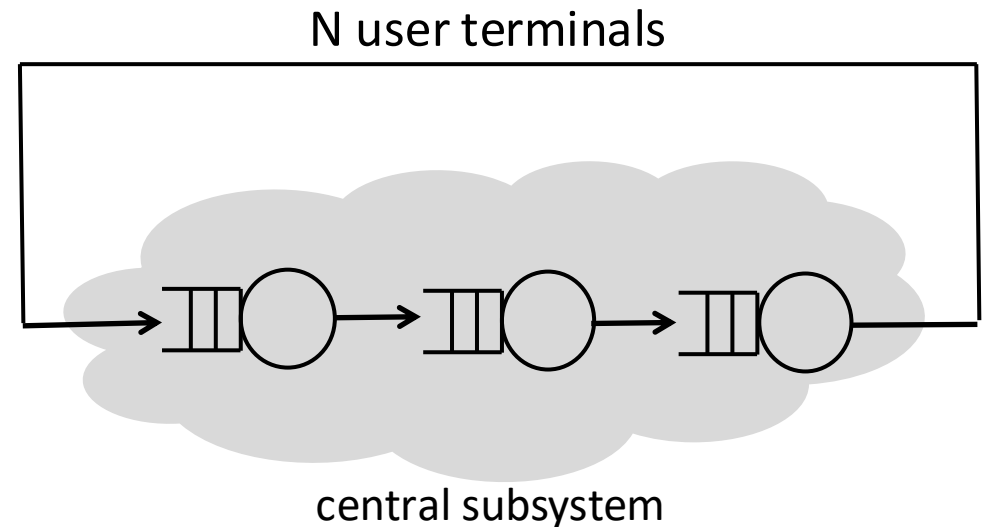
Closed Queueing Networks

- No external arrivals and departures
 - i. interactive (terminal-driven) systems
 - ii. batch systems

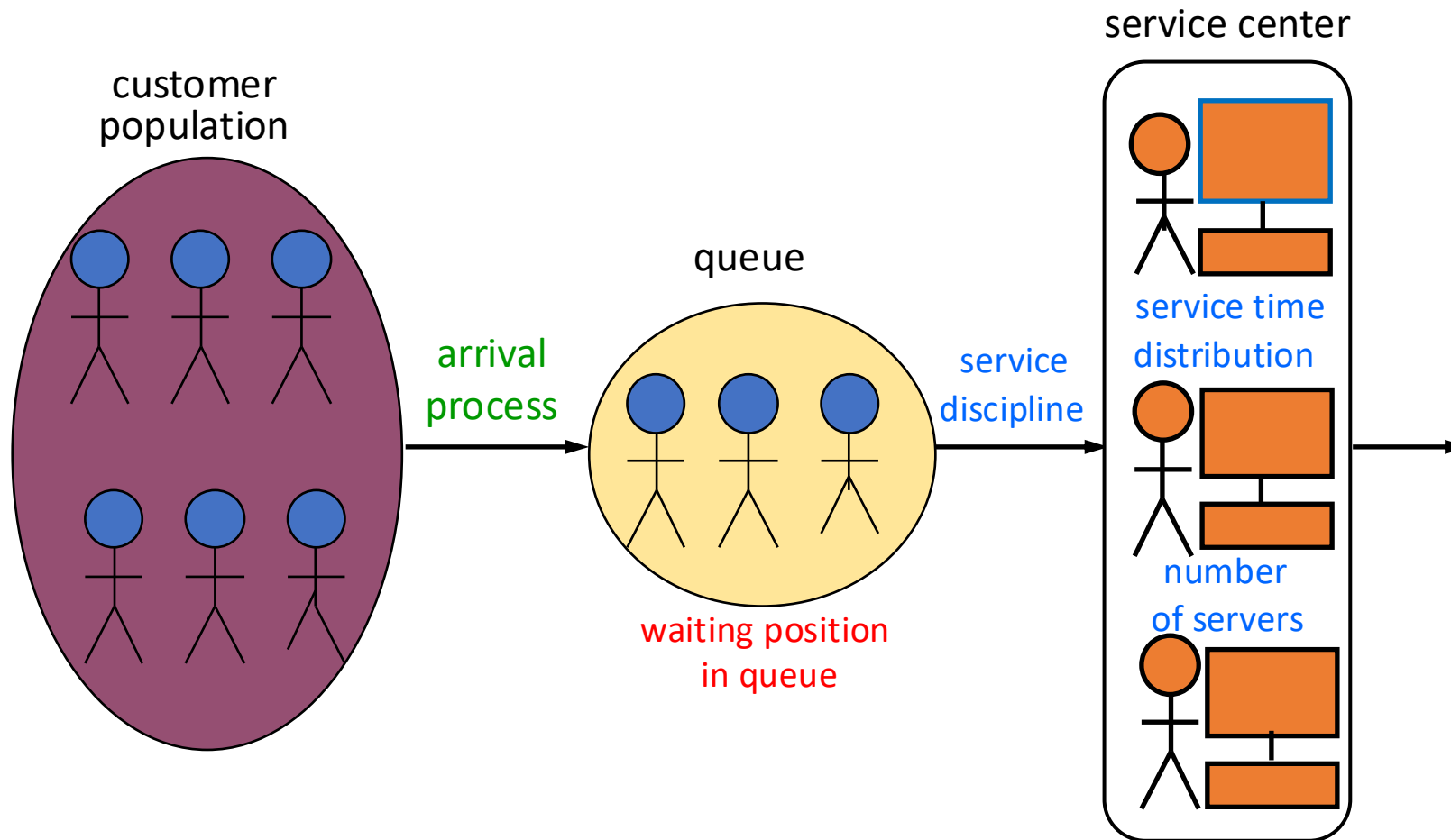


Batch Systems

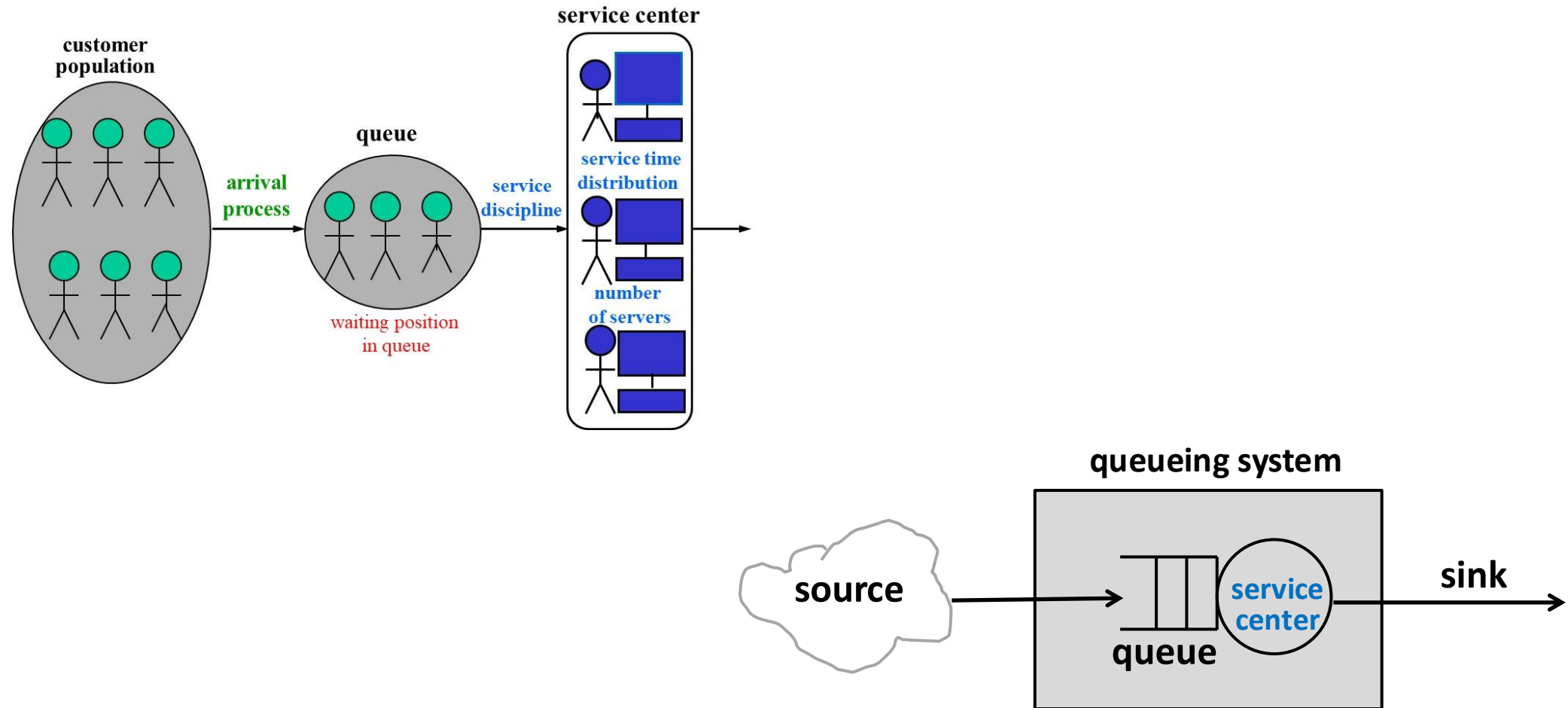
- Run as many jobs as possible overnight, i.e., achieve high throughput
- As one job completes another one is started and thus there is always N jobs in the subsystem
- N is limited by the number of jobs that can fit into the computer memory



Basic Components of a Queueing System



Basic Components of a Queueing System



Practical Example: one queue or one queue per worker (server)

Queue organisation affects load balancing and waiting time

One Shared Queue

All requests wait in one queue
The next available worker takes the next request

Often improves load balancing when workers are similar

Separate Queues

Each worker has its own queue
Requests must be assigned to a worker

Can create imbalance when request sizes vary

Basic Components of a Queueing System

- **source**: customer arrivals from a **source** may be **infinite** or **finite**
 - when a customer source is finite but very large, it is common to assume an infinite source - approximation simplifies method to solve the problem
- **arrival process**: if customers arrive at times $t_1, t_2, \dots, t_j$, the random variable $\tau_j = t_j - t_{j-1}$ is the **inter-arrival time**
- **service time distribution**: time each customer spends at the service center is the **service time**
- **number of servers**: single server systems - **one server** serves a single customer at a time, or multiple server systems (**m servers**), all of identical or non-identical capabilities and serves m customers at once
- **service discipline** or **queue discipline**: order that customers is serviced
 - e.g., First-come-first-server (FCFS), Round-Robin (RR), Processor Sharing (PS), Shortest Processing Time first (SPT), ...

Kendall Notation (1953)

British Statistician – David G. Kendall

- describes characteristics of common queueing systems
- Kendall notations:

$A/S/m/K/N/SD$

where

- A is the inter-arrival time distribution
- S is that service time distribution
- m is the number of servers
- K is the system's capacity (number of places in the queue + number of servers)
- N is the number of customers in the source or population size
- SD is the service discipline

Kendall Notation

- Inter-arrival and service time distributions are denoted by a one-letter symbol:
 - D for deterministic (implies that times are constant and there is no variance)
 - M for exponential (or *memoryless* distribution)
 - E_k for Erlang-k
 - H_k for hyper-exponential (with k stages)
 - G for general (any probability distribution)
- A shorthand notation **A/S/m** is commonly used
 - no limit on the length of the queue
 - source is infinite
 - service discipline is FCFS

Practical Example: Kendall Notation

M/M/1

One worker
Poisson arrivals
Exponential service time
Example: a simplified single-thread service

M/G/4

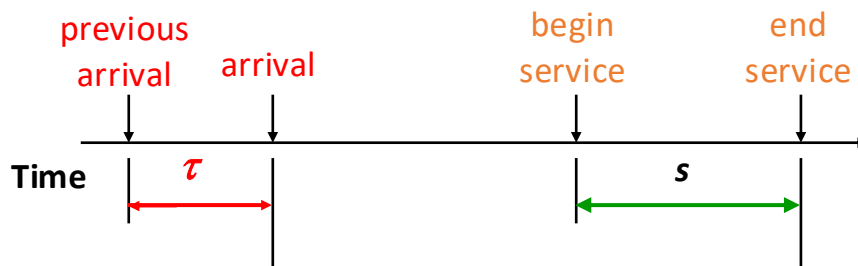
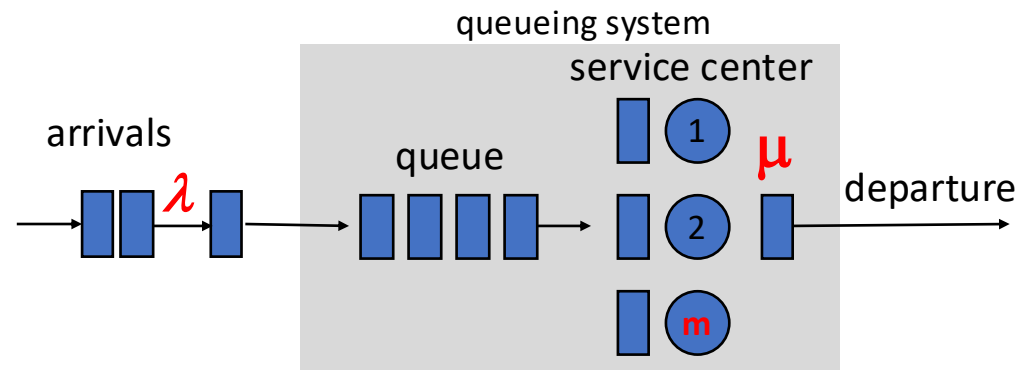
Four workers
Poisson arrivals
General service-time distribution
Useful when request sizes vary

D/D/10/100

Ten workers
Deterministic arrivals and service
Maximum of 100 customers in the system

Notation is a modeling choice, not a measurement result
Validate the assumptions against the workload

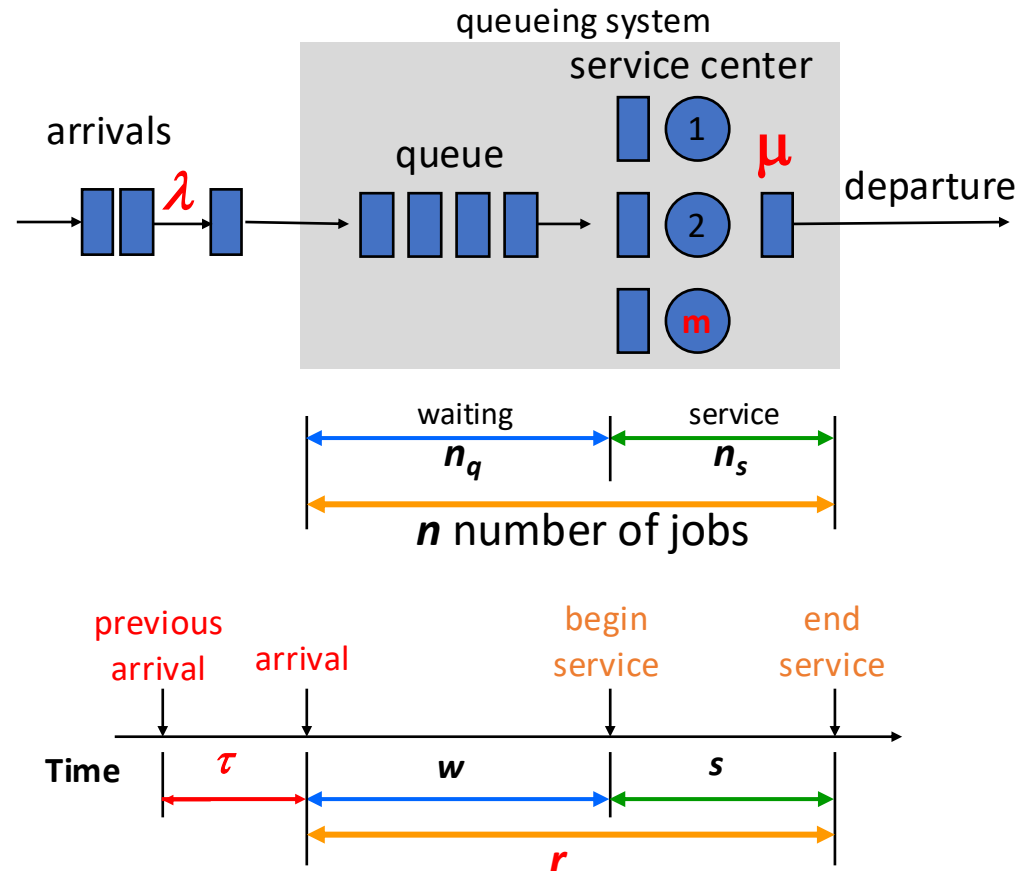
Random Variables for All Queues



Consider an arbitrary arrival stream of customers $c_j, j=1,2,\dots$

- t_j = arrival epoch of c_j , $t_0 \equiv 0$ & $t_0 \leq t_1 \leq t_2 \dots$
- $\tau_j = t_j - t_{j-1}$ = inter-arrival time, i.e., time between two successive arrivals
- λ = mean arrival rate, can be a function of the state of the system
- s = service time per job
- μ = mean service rate per server, total service rate for m servers is $m\mu$

Random Variables for All Queues



Number of Jobs

- n (number of jobs in the system at time t) = $n_q + n_s$
- n_q = number of jobs waiting to receive service
 - This is always less than n , since it does not include the jobs currently receiving service.
- n_s = number of jobs receiving service
- $E[n]$ (mean number of jobs in the system) = $E[n_q] + E[n_s]$

Response Time

- r (response time or the time in the system) = $w + s$
 - This includes both the time waiting for service and the time receiving service.
- w = waiting time or the time interval between arrival time and the instant the service begins
- $E[r]$ (mean response time in the system) = $E[w] + E[s]$
- $E[s]$ (mean service time)
- $E[w]$ (mean waiting time)

Rules for all Queues ($G/G/m$)

1. Stability condition: a system is stable when its long-run behaviour remains bounded

- the mean arrival rate should be less than the mean service rate

$$\lambda < m\mu$$

- Note that finite population and finite buffer systems are always stable
- Condition is necessary for a bounded queue, but does not guarantee an acceptable response time

2. Number in system versus number in queue:

$$n = n_q + n_s$$

$$E[n] = E[n_q] + E[n_s]$$

3. Time in system versus time in queue:

$$r = w + s$$

$$E[r] = E[w] + E[s]$$

4. Number of jobs versus response time (aka **Little's Law**):

$$E[n] = \lambda E[r]$$

Practical Example: Checking Stability

Use the stability condition as an early capacity warning

Current System

Four workers
Each worker serves $\mu = 100$ requests/s
Total service capacity = 400 requests/s

Current Demand

Arrival rate $\lambda = 360$ requests/s
The system is stable
There is limited headroom

Traffic Increase

Arrival rate $\lambda = 450$ requests/s
 $\lambda > m\mu$
The queue grows unless demand or capacity changes

Stability is necessary for a bounded long-run queue, but it does not guarantee an acceptable response time

Little's Law

1. The mean number of jobs in the system is the product of the mean arrival rate and mean response time

$$E[n] = \lambda E[r]$$

2. The mean number of jobs in queue is the product of the mean arrival rate and mean waiting time

$$E[n_q] = \lambda E[w]$$

3. The mean number of jobs receiving service is the product of the mean arrival rate and mean job service time

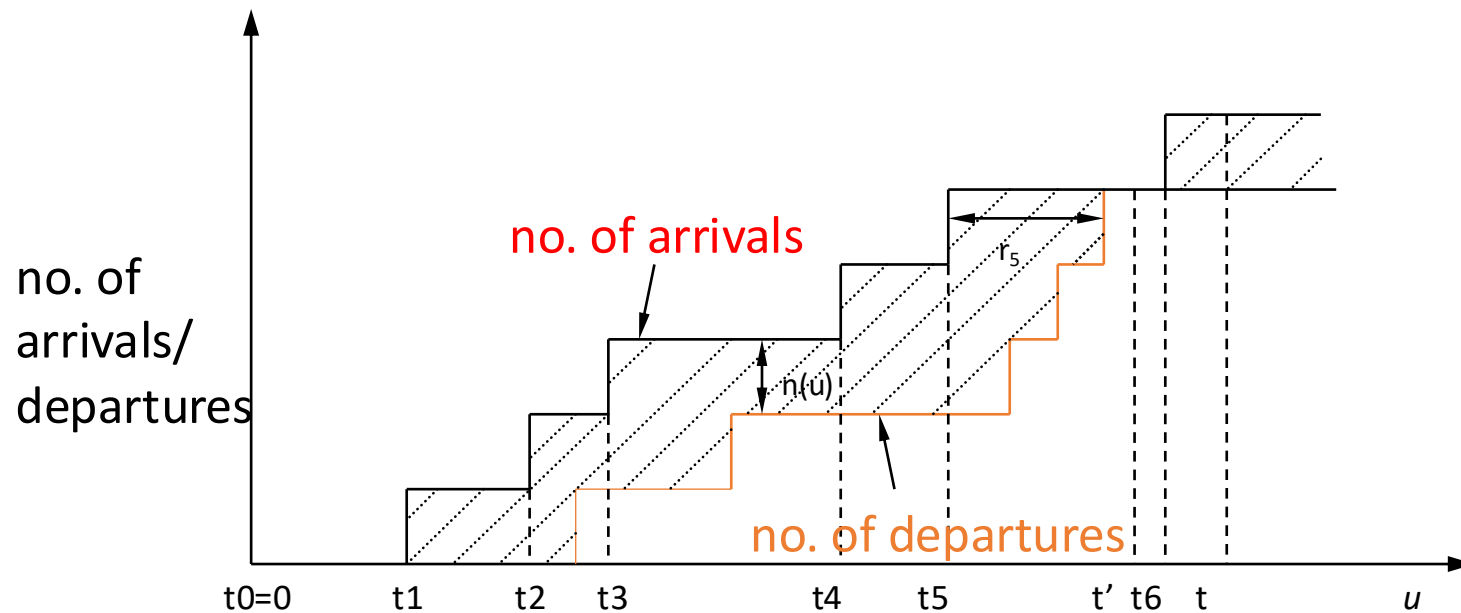
$$E[n_s] = \lambda E[s]$$

- Applies to any **stable system** as long as the **number of jobs entering the system is equal to the number of jobs completing the service**

Little's Law (J.D.C. Little, 1961)

$$E[n] = \lambda E[r]$$

Proof: a graphical approach



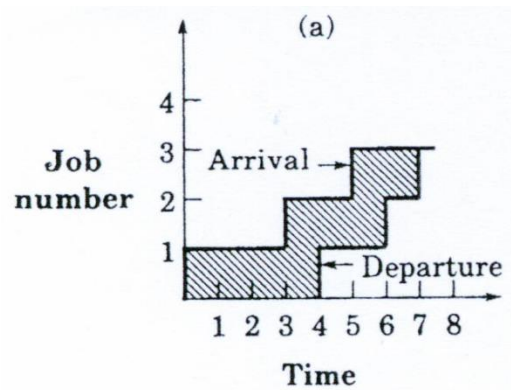
Sample path showing the number of arrivals and departures and an empty system at $t_0=0$

Proof: $E[n] = \lambda E[r]$

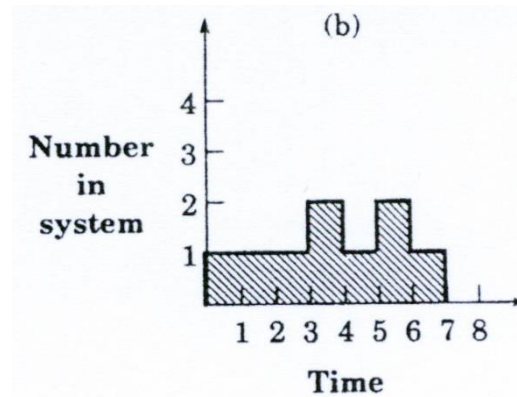
- Suppose we observe the system for a **time interval T** and record the arrival and departure times of each job. If T is large, the number of arrivals (N) would be equal to the number of departures (N). Thus,

$$\text{Arrival rate } (\lambda) = N/T$$

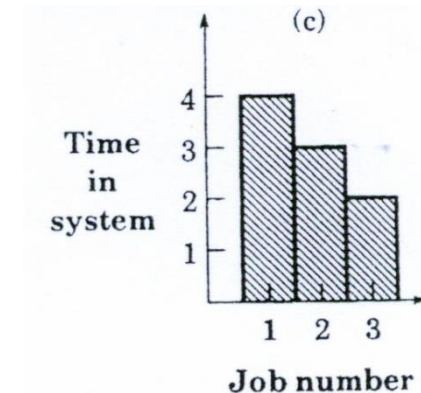
- 3 ways to plot the data:



(a) no. of arrivals & departures over time



(b) no. of jobs at an instance over time



(c) time each job spent in system - subtract arrival from departure time gives time in system

Proof: $E[n] = \lambda E[r]$

Let J be the shaded area (all 3 shaded areas are equal).

From graph c, the *mean time spent* in the system

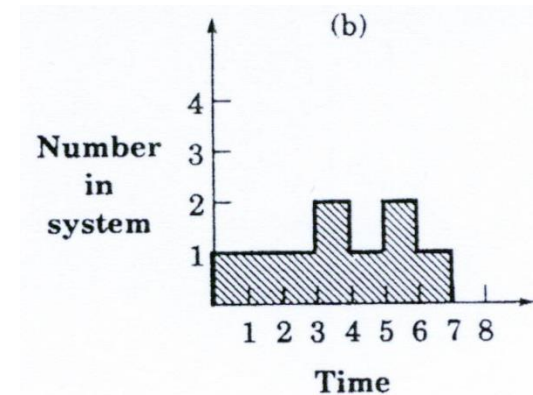
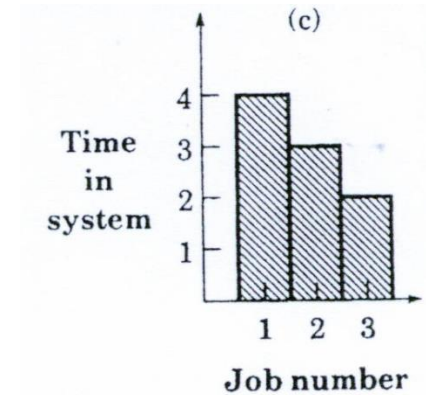
$$E[r] = J/N$$

From graph b, the *mean number of jobs* in the system

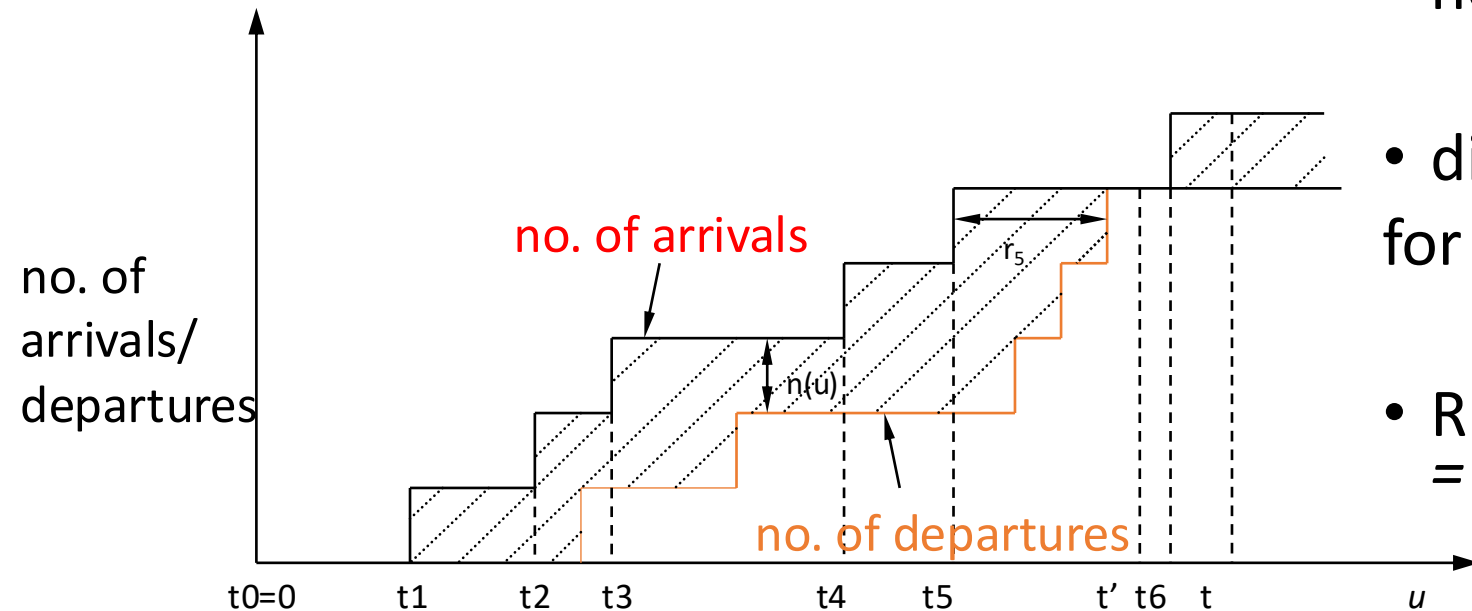
$$\begin{aligned} E[n] &= J/T \times N/N \\ &= N/T \times J/N \end{aligned}$$

Thus,

$$E[n] = \lambda E[r] \quad \text{Little's Law}$$



Proof: $E[n] = \lambda E[r]$



- note that the shaded area till t' is

$$\int_0^{t'} N(u) du = \sum_{j=1}^{\Lambda(t')} T_j$$

- however, the shaded area till t is

$$\int_0^t N(u) du = \sum_{j=1}^{\Lambda(t)} T_j + \varepsilon \quad (\text{error term})$$

- divide the above relation by t and

for $t \rightarrow \infty$, i.e.,

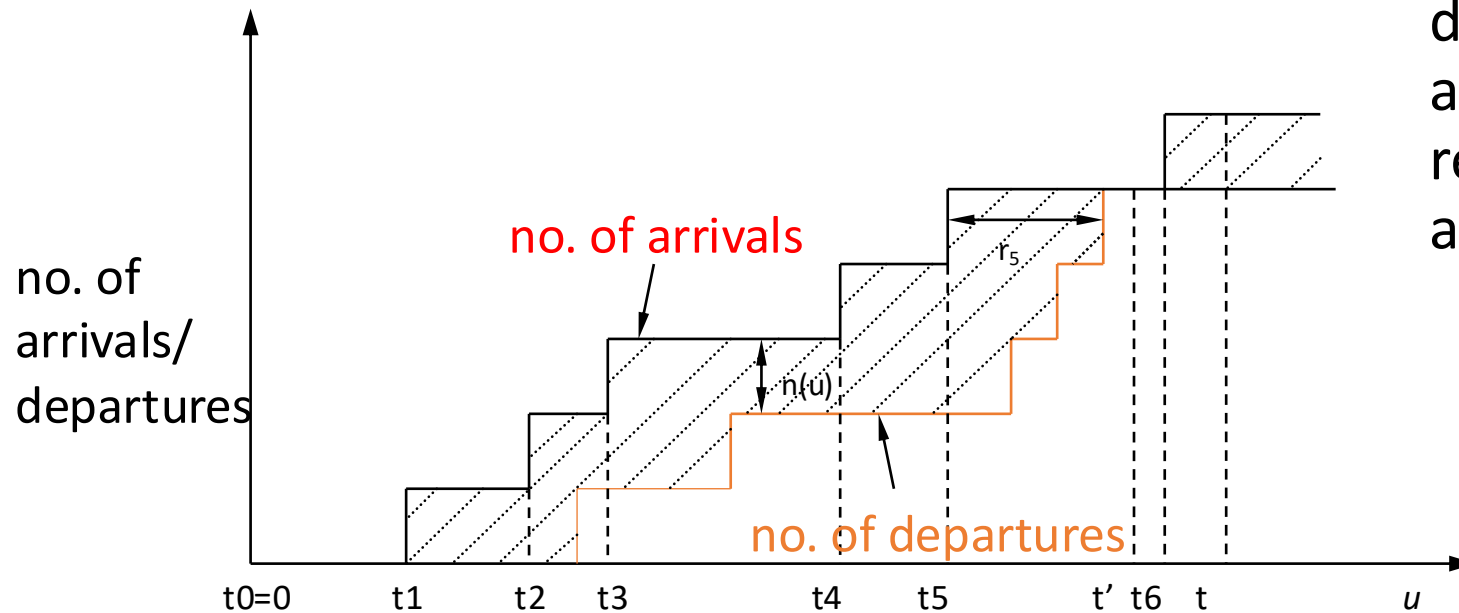
$$\lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t n(u) du \cong \lim_{t \rightarrow \infty} \frac{1}{t} \sum_{j=1}^{\Lambda(t)} r_j$$

- RHS reduces to λr , therefore, we have $n = \lambda r$, and hence

$$E[n] = \lambda E[r]$$

Proof: $E[n_q] = \lambda E[w]$

- Little's Law is independent of the service discipline, inter-arrival distribution, service-time distribution, and number of queues
- It applies when the system has well-defined boundaries, long-run average arrivals and departures balance, and the relevant averages are finite (e.g. stable and steady state)



Application of Little's Law

- Little's Law is independent of the service discipline, inter-arrival distribution, service-time distribution, and number of queues
 - It can be applied to a subsystem, but the arrival and departure definitions must refer to the same boundary
- The standard queueing form $E[n] = \lambda E[r]$ applies when the system has well-defined boundaries, long-run average arrivals and departures balance, and the relevant averages are finite (e.g. **stable** and **steady state**)
- In an **unstable** system, the queue may grow without bound, so $E[n]$ and $E[r]$ may not converge to finite values
 - Little's Law can still be used over a finite observation interval, but boundary effects must be considered (not covered in CS5239)
- Note:
 - **Stability** prevents the queue from growing without bound
 - **Steady state** means the measured averages have settled into a long-run regime

Example 1: Little's Law

If we monitor a disk server for a period of time and based on measurement average time to satisfy an I/O request is 100 milliseconds. The I/O rate was about 100 requests per second.

What was the mean number of requests at the disk server?

Applying Little's law:

$$E[n] = \lambda E[r]$$

- mean number in the disk server = I/O arrival rate x I/O response time
= (100 requests / second) x (0.1 second)
= 10 requests

Traffic Intensity & Utilization

- Traffic intensity ρ
 - If $\rho < 1$, the system is said to be in steady state:
 - $\rho = \lambda/\mu$ (1 server)
 - % measure of load on the overall system
 - e.g. “Traffic intensity of 50%”
 - If $\rho \geq 1$, the system cannot keep up with demand (a lot of our math breaks down in this situation)
- Utilization U
 - The proportion of time the server is busy
 - Maxes out at 100%, or saturated
 - If jobs are never lost, $U = \rho$
 - If jobs can be lost, $U \leq \rho$

Traffic Intensity (one server)- ρ

When jobs are not lost and the system is in steady state

$$\rho = \lambda E[s] = E[n_s]$$

- Server is idle
 - P_0 – probability that server is idle = $1 - \rho = 1 - \lambda E[s]$
 - If the number of jobs in a system grows continuously and becomes infinite, the system is said to be unstable
- Proof:
 $\rho = \lambda/\mu$ where $\mu = 1/E[s]$
 - From Little's law: $E[n_s] = \lambda E[s]$
 - Therefore: $\rho = \lambda E[s] = E[n_s]$
 - One server: $\rho = E[n_s]$ and $\rho < 1$ for a stable system

Example 2: Little's Law

- Consider a server system with one queue and four **workers**
- Suppose we've been running this system for a while and have observed an average number of ten tasks in the system (queueing). This number represents the number of tasks in the system, $E[n]$.
- In our measurements we've observed that each **worker** can process on average 10 tasks per second, giving us mean throughput, λ_{worker}
- How long does it take the average task to move through the system?

$$\lambda = 4 \lambda_{\text{worker}}$$

$$E[r] = E[n]/\lambda = 10/40 \text{ seconds} = 0.25 \text{ seconds}$$

Example 2 (cont)

- Suppose we expect the average number of tasks arriving in the queue to increase from 10 to 50 due to an upcoming Black Friday sale.
- What effect would this have on response time?
 $E[r]' = 50/40 \text{ seconds} = 1.25 \text{ seconds}$
- What effect would adding one more worker to our system have?
 $E[r]'' = 50/50 \text{ seconds} = 1 \text{ second}$
- We've been striving to keep response times below a half of a second. How fast should the workers become to achieve this?
 - $E[r] = 0.5 \text{ seconds} \Rightarrow \lambda_{\text{worker}} = 20 \text{ tasks per second}$ ($E[n] = 50$ with 5 workers)

Basics of Queue (1)

1. Parameters:
 - λ = arrival rate in jobs per unit time
 - μ = service rate in jobs per unit time
 - m = number of servers
2. For a system to be stable, $\lambda < m\mu$
3. Mean number of jobs in the system: $E[n] = E[n_q] + E[n_s]$
4. Mean response time: $E[r] = E[w] + E[s]$
5. Little's law: $E[n] = \lambda E[r]$
6. Mean number of jobs in the queue: $E[n_q] = \lambda E[w]$
7. Mean number of jobs receiving service: $E[n_s] = \lambda E[s]$
8. Traffic intensity (one server): $\rho = \lambda E[s] = E[n_s]$
9. Probability that server is idle: $P_0 = 1 - \rho = 1 - \lambda E[s]$

Basics of Queue (2)

- $E[n]$ (mean number of jobs in the system) = $E[n_q] + E[n_s]$
where n_q = number of jobs waiting to receive service, and n_s = number of jobs receiving service
- $E[r]$ (mean response time in the system) = $E[w] + E[s]$
where w = waiting time of a job, and s = service time per job
- Little's Law:
 - mean number of jobs in system = mean arrival rate $\times$ mean response time
i.e. $E[n] = \lambda E[r]$
 - mean number of jobs in queue = mean arrival rate $\times$ mean waiting time
i.e. $E[n_q] = \lambda E[w]$

Little's Law: relates the number of jobs and time

Summary

- Stochastic processes
- Basics of queueing system
- Kendall notation
- Little's law

Key Terms

- open versus closed queueing networks
- interactive systems, batch systems
- inter-arrival time
- arrival rate, service rate
- Markov process
- Poisson process

Reading

- [Jain] – chapter 30